# Python Activity-Based Learning: Case Studies 1 to 10

**Syllabus Scope:** Computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditionals, loops, strings, patterns, lists, nested lists, tuples, unpacking.

## 1. University Scholarship Allocation

**Problem Statement:** Evaluate applicants using aggregate marks, subject minimums, attendance, and income; rank eligible students and resolve ties.

### Algorithm & Pseudocode

**Algorithm:** 1. Input list. 2. Filter eligibility criteria. 3. Compute score. 4. Sort descending.

```
FOR each applicant IN data:
  IF agg >= 85 AND att >= 90 AND inc <= 300000:
    calc score, add to eligible
SORT eligible BY score DESC
```

### Professional Flowchart




### Python Solution & Test Cases

```
def allocate_scholarships(data):
    eligible = [ (agg*0.7+att*0.3, inc, name) for name, agg, min_s, att, inc in data if agg >= 85 and m
in_s >= 75 and att >= 90 and inc <= 300000 ]
    eligible.sort(key=lambda x: (-x[0], x[1]))
    return [item[2] for item in eligible]

students = [("Alice", 88, 80, 92, 250000), ("Bob", 90, 85, 95, 350000), ("Charlie", 86, 78, 91, 20000
0)]
print("Ranked Scholars:", allocate_scholarships(students))
# Output: ['Charlie', 'Alice']
```

## 2. Progressive Electricity Billing

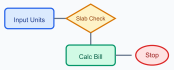
**Problem Statement:** Calculate household bills using consumption slabs, fixed charges, surcharges, and rebates, comparing across months.

### Algorithm & Pseudocode

**Algorithm:** 1. Input units. 2. Evaluate slab ranges. 3. Add fixed charges & tax. 4. Check spikes.

```
IF units <= 100: bill = units * 3.0
ELSE IF units <= 200: bill = 300 + (units - 100) *
5.0
ELSE: bill = 800 + (units - 200) * 8.0
ADD fixed_charge; CHECK spike
```

### Professional Flowchart




### Python Solution & Test Cases

```
def calc_bill(units):
    if units <= 100: b = units * 3.0
    elif units <= 200: b = 300 + (units - 100) * 5.0
    else: b = 800 + (units - 200) * 8.0
    return b + 50.0

months_units = [90, 150, 280]
bills = [calc_bill(u) for u in months_units]
print("Monthly Bills:", bills)
print("Spike detected:", bills[2] > 1.3 * bills[1])
# Output: [320.0, 600.0, 1440.0], Spike: True
```

## 3. Hospital Billing and Patient Classification

**Problem Statement:** Process consultation, diagnostic, and treatment charges, apply conditional concessions, and classify patients.

### Algorithm & Pseudocode

**Algorithm:** 1. Input charges and category. 2. Sum subtotal. 3. Apply concession. 4. Classify tier.

```
total = consult + diag + treat
IF category == "Senior": total *= 0.80
ELSE IF category == "General": total *= 0.95
CLASSIFY tier BASED ON total
```

### Professional Flowchart




### Python Solution & Test Cases

```
def process_hospital_bill(consult, diag, treat, category):
    total = consult + diag + treat
    if category == "Senior": total *= 0.80
    elif category == "Child": total *= 0.85
    tier = "Platinum" if total > 15000 else "Standard"
    return round(total, 2), tier

print(process_hospital_bill(1000, 4500, 12000, "Senior"))
# Output: (14000.0, 'Standard')
```

## 4. Employee Payroll and Deduction Analysis

**Problem Statement:** Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions.

### Algorithm & Pseudocode

**Algorithm:** 1. Input basic, allowance, overtime. 2. Compute gross. 3. Apply tax brackets. 4. Compute net.

```
gross = basic + allowances + ot_pay
IF gross > 100000: tax = gross * 0.20
ELSE: tax = gross * 0.10
net = gross - tax - pf
```

### Professional Flowchart




### Python Solution & Test Cases

```
def calculate_payroll(basic, hra, da, ot_hrs, inc):
    gross = basic + hra + da + (ot_hrs * 250) + inc
    tax = gross * 0.20 if gross > 100000 else (gross * 0.10 if gross > 50000 else 0)
    net = gross - tax - (basic * 0.12)
    return {"Gross": gross, "Tax": tax, "Net": net}

print(calculate_payroll(45000, 15000, 10000, 10, 5000))
# Output: {'Gross': 77500.0, 'Tax': 7750.0, 'Net': 64350.0}
```

## 5. Multi-Plan Telecommunications Cost Optimization

**Problem Statement:** Evaluate mobile plans against usage, validity, and budget constraints; identify the most economical eligible plan.

### Algorithm & Pseudocode

**Algorithm:** 1. Input usage profile. 2. Iterate plan tuples. 3. Check inclusions. 4. Find min cost.

```
FOR each plan IN plans:
  IF plan.data >= req_data AND plan.calls >= req_c
alls:
    cost = plan.price + extra_usage
    IF cost < min_cost: best_plan = plan
```

### Professional Flowchart




### Python Solution & Test Cases

```
def optimize_plan(plans, req_calls, req_data, budget):
    best_plan, min_cost = "None", float('inf')
    for name, price, calls, data in plans:
        if calls >= req_calls and data >= req_data and price <= budget and price < min_cost:
            min_cost, best_plan = price, name
    return best_plan, min_cost

plans_data = [("PlanA", 399, 1000, 15), ("PlanB", 599, 3000, 30), ("PlanC", 299, 500, 5)]
print(optimize_plan(plans_data, 800, 10, 500))
# Output: ('PlanA', 399)
```

## 6. Railway Fare and Reservation Calculator

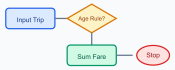
**Problem Statement:** Calculate fares for multiple passengers using distance slabs, travel classes, age concessions, and cancellation penalties.

### Algorithm & Pseudocode

**Algorithm:** 1. Input distance & class. 2. Iterate passengers. 3. Apply age rules. 4. Sum fees.

```
FOR age IN passengers:
  fare = distance * rate * multiplier
  IF age >= 60: fare *= 0.50
  TOTAL += fare + res_fee
```

### Professional Flowchart




### Python Solution & Test Cases

```
def calculate_railway_fare(distance, class_type, ages):
    multiplier = {"AC1": 2.5, "AC2": 2.0, "Sleeper": 1.0}[class_type]
    total = sum((distance * 1.2 * multiplier * (0.5 if age >= 60 else (0 if age <= 5 else 1)) + 30) for
age in ages)
    return round(total, 2)

print(calculate_railway_fare(650, "AC2", [25, 62, 4]))
# Output: 3390.0
```

## 7. Retail Billing with Multiple Discount Rules

**Problem Statement:** Process multiple products, quantities, category taxes, threshold discounts, and maximum limits for itemized invoices.

### Algorithm & Pseudocode

**Algorithm:** 1. Input cart items. 2. Calculate item taxes. 3. Evaluate threshold discounts. 4. Print total.

```
FOR item IN cart:
  subtotal += (price * qty) * (1 + tax_rate)
IF subtotal > 3000: discount = subtotal * 0.10
```

### Professional Flowchart




### Python Solution & Test Cases

```
def generate_retail_invoice(cart):
    subtotal = sum(price * qty * (0.18 if cat == "Electronics" else 1.05) for _, price, qty, cat in car
t)
    discount = subtotal * 0.10 if subtotal > 3000 else 0
    return round(subtotal - discount, 2)

cart_items = [("Laptop", 45000, 1, "Electronics"), ("Notebook", 50, 5, "Stationery")]
print(generate_retail_invoice(cart_items))
# Output: 47287.5
```

## 8. Manufacturing Cost and Profitability Analysis

**Problem Statement:** Calculate material costs, labour charges, wastage, machine costs, and selling prices; determine profit targets.

### Algorithm & Pseudocode

**Algorithm:** 1. Input material, labour, wastage. 2. Calculate total cost. 3. Check profit target.

```
total_cost = mat + labour + wastage + machine
profit = selling_price - total_cost
meets_target = profit >= target
```

### Professional Flowchart




### Python Solution & Test Cases

```
def analyze_profitability(mat, hours, rate, waste_pct, machine, selling_price, target):
    total_cost = mat + (hours * rate) + (mat * waste_pct / 100) + machine
    profit = selling_price - total_cost
    return {"Total Cost": total_cost, "Profit": profit, "Meets Target": profit >= target}

print(analyze_profitability(1200, 15, 200, 5, 500, 6000, 1000))
# Output: {'Total Cost': 4760.0, 'Profit': 1240.0, 'Meets Target': True}
```

## 9. Multi-Currency Travel Budget Analysis

**Problem Statement:** Convert travel expenses using exchange rates, incorporate conversion fees, compare against budget, and find largest expense.

### Algorithm & Pseudocode

**Algorithm:** 1. Input expenses. 2. Convert currencies & add fee. 3. Sum total & find max category.

```
FOR cat, amt, curr IN expenses:
  conv = amt * rates[curr] * 1.02
  total += conv; TRACK max
```

### Professional Flowchart




### Python Solution & Test Cases

```
def analyze_travel_budget(expenses, rates, budget):
    total = sum(amt * rates[curr] * 1.02 for _, amt, curr in expenses)
    max_cat = max(expenses, key=lambda x: x[1] * rates[x[2]])[0]
    return {"Total": round(total, 2), "Largest Expense": max_cat, "Over Budget": total > budget}

exps = [("Hotel", 500, "USD"), ("Food", 300, "EUR"), ("Transport", 15000, "INR")]
rates = {"USD": 83.0, "EUR": 90.0, "INR": 1.0}
print(analyze_travel_budget(exps, rates, 100000))
# Output: {'Total': 72914.0, 'Largest Expense': 'Hotel', 'Over Budget': False}
```

## 10. Campus Event Financial Planning

**Problem Statement:** Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions and break-even count.

### Algorithm & Pseudocode

**Algorithm:** 1. Input fixed & variable costs. 2. Calculate deficit. 3. Compute break-even count.

```
total_cost = fixed + (var_cost * count)
deficit = total_cost - sponsorship
breakeven = fixed / (reg_fee - var_cost)
```

### Professional Flowchart




### Python Solution & Test Cases

```
def event_financial_planning(fixed, var_cost, reg_fee, sponsorship, count):
    total_cost = fixed + (var_cost * count)
    net_deficit = total_cost - sponsorship
    breakeven = fixed / (reg_fee - var_cost) if reg_fee > var_cost else float('inf')
    return {"Total Cost": total_cost, "Net Deficit": net_deficit, "Breakeven Count": int(breakeven) +
1}

print(event_financial_planning(25000, 300, 800, 10000, 50))
# Output: {'Total Cost': 40000, 'Net Deficit': 30000, 'Breakeven Count": 50}
```